

Informe de evaluación

Tarea final — Curso 1: un agente para su proyecto

Proyecto evaluado	Cotizador de exámenes médicos (V1, banco de pruebas)
Ubicación	cotizador-opencode/cotizador-examenes/
Documento base	docs/tarea_final.pdf (pauta de la tarea final, 4 páginas)
Método	Revisión de código, especificación, prompts, datos, pruebas y la corrida del notebook
Fecha	05 de octubre de 2026

1. Resumen ejecutivo

El proyecto es de **calidad alta y está muy completo**. Cubre la base obligatoria con holgura (LLM real, ciclo ReAct con parada explícita, historial por checkpoint, seguridad y privacidad) y declara cuatro ampliaciones que suman el tope de +3,0: **RAG sobre el Redis del curso, workflow con router, evaluación con golden set y guardrail/juez (verificador)**. La arquitectura está separada por capas, los prompts son un archivo por llamada y hay pruebas de validación versionadas.

El principal riesgo **no es de funcionalidad sino de evidencia**. El notebook entregado y ejecutado demuestra **un solo turno**, aunque la tarea exige mostrar el segundo turno del historial; y las corridas del golden set y de seguridad **no están versionadas** (no hay archivos resultados_*.xlsx ni seguridad_*.xlsx), por lo que el propio checklist del notebook las marca como «PENDIENTE». El código que las produce existe y es reproducible, pero el revisor no puede ver el resultado sin ejecutarlas.

Bloque	Máximo	Estimado	Cumplimiento
Base obligatoria (inicial + 5 criterios)	4,0	3,85	96 %
Ampliaciones declaradas (4 bonos)	+3,0	+2,65	88 %
Total	7,0	6,50	93 %

Lectura del puntaje. La nota estimada asume que el revisor ejecuta las pruebas declaradas y que pasan, como afirma el plan del proyecto. Si se califica *solo con la evidencia versionada hoy* (sin correr nada), el golden set (0,5) y parte del guardrail quedarían sin respaldo observable y el resultado bajaría a ~86 % (**6,0/7,0**). Los ajustes del capítulo 8 permiten aspirar al **100 % (7,0/7,0)**.

2. Alcance y método

Se revisó el árbol completo del proyecto activo, el plan.json, la especificación (spec.md) y sus anexos de revisión, los siete prompts del agente, las cuatro herramientas, los nodos del grafo, el simulador, la ingesta y los arneses de validación. Se extrajo el texto de la pauta y se leyó la corrida ejecutada dentro del notebook de entrega.

Limitación honesta: no se ejecutó el agente en vivo porque el entorno de revisión no dispone de credenciales ni del Redis del curso. La evaluación se apoya en tres evidencias: (a) el código y sus validaciones, que compila sin errores; (b) los documentos y pruebas versionados; y (c) la única corrida real registrada, la del notebook. Por eso el veredicto se expresa como *estimación de cumplimiento* y no como nota definitiva.

3. Detalle del puntaje por criterio

Criterio (base obligatoria)	Máx	Est.	Evidencia principal
Inicial (flujo de punta a punta)	1,0	1,0	Grafo compilado START->END; corrida del notebook termina en «verificador».
Caso y criterio de éxito	0,5	0,5	Usuario, entrada, alcance y salida definidos en spec e intent; verdad del simulador campo por campo.
LLM real y trazabilidad	0,5	0,5	ChatOpenAI (OpenAI/OpenCode Zen), ficha de modelo y parámetros, 7 prompts, traza narrativa del notebook.
ReAct integrado	1,0	1,0	Bucle agente<->herramientas con observación devuelta como ToolMessage; parada explícita en despues_agente y tope 10.
Historial simple	0,5	0,4	Checkpoint + router que reusa ciudad/previsión; caso G04 ejerce 2 turnos. PERO el notebook entregado solo ejecuta 1 turno.
Seguridad básica	0,5	0,45	BLOQUE_UNIVERSAL/AGENTE en cada llamada que decide; RUT enmascarado, síntomas, inyección; S1-S6 definidas.

Subtotal base: 3,85 / 4,0 (96 %).

Bono declarado (spec §9, DA-06)	Máx	Est.	Evaluación
RAG (Redis del curso)	+1,0	0,9	Implementado en consultar_documentos: embeddings OpenAI (768 dims), índice RediSearch nombre/prefijo, campos, coseno, algoritmo FLAT, top-k=4. Decisión visible por elección de herramienta.
Workflow adicional (router)	+1,0	1,0	Router clasifica 4 rutas y main.ruta_condicional decide; la traza y el golden set verifican la ruta elegida.
Evaluación con golden set	+0,5	0,35	golden_set.json G01-G16 completo y versionado + run_golden.py que produce Detalle/Resumen/Fallos. Falta la corrida versionada: no hay .xlsx de resultados.
Guardrail / juez (verificador)	+0,5	+0,4	Nodo de código independiente que retira recomendaciones y marca sin respaldo como «no confirmado»; la corrida del notebook muestra una corrección aplicada.

Subtotal ampliaciones: +2,65 / +3,0 (88 %).

4. Base obligatoria: análisis por criterio

Caso y criterio de éxito (0,5) — cumple

El producto tiene usuario (persona en Chile que necesita un examen), entrada (lenguaje natural), alcance (informa, no recomienda, no agenda, no pide datos personales) y salida esperada (opciones comparables). El caso justifica el LLM (interpretar necesidad en palabras propias, distinguir exámenes por propósito, simular la llamada) y el uso de herramientas (búsqueda, identificación, consulta, RAG). El criterio observable es fuerte: el golden set compara campo por campo contra la **verdad del simulador** (ruta, solicitud, comparables, dudosos, descartados, motivo de parada). *Observación:* la portada del notebook no enuncia el caso ni el criterio de éxito, aunque spec §7.7 lo pide; conviene añadirlo.

LLM real y trazabilidad (0,5) — cumple

Las llamadas son reales por API (services/llm.py con ChatOpenAI; proveedor openai u OpenCode Zen vía base_url). El modelo y la configuración están documentados en el README (tabla de ficha: proveedor, modelo, temperaturas 0,0/0,1, timeout 90 s, reintentos 1, N=2, K=6, tope 10) y en config.py, único lector del entorno. Cada llamada al LLM tiene su prompt en prompts/. La trazabilidad se ve en la narración del notebook y en la captura de paso a paso de run_golden.py (astream con stream_mode=updates).

ReAct integrado (1,0) — cumple

El nodo agente pide herramientas; el nodo herramientas valida en código, ejecuta y devuelve la observación como ToolMessage que vuelve al LLM; main.despues_agente decide repetir o consolidar; la parada es explícita (N comparables, sin centros aplicables o tope) y el tope MAX_ITERACIONES=10 está aplicado en código. La corrida del notebook registra 15 pasos del grafo y 5 rondas de 10. Es un ReAct completo y acotado.

Historial simple (0,5) — cumple en código; evidencia incompleta

Es correcto: hay MemorySaver como checkpoint, el router recibe el historial visible y su prompt ordena reutilizar ciudad y previsión del turno anterior, y prevision_preguntada persiste para no repetir la pregunta. No hay respuesta fija: la reutilización es una decisión del LLM sobre el historial enviado. Además, el caso G04 del golden set ejercita dos turnos (turno 2: «No sé») donde la ciudad debe reusarse. **El problema es de entrega:** el notebook ejecutado contiene un único turno («1 turnos en el mismo hilo»), justo lo que la pauta pide demostrar en un segundo turno. Es el riesgo más alto porque toca el tope de 3,0 si el revisor no lo ve funcionar.

Seguridad básica (0,5) — cumple

Cada llamada que decide o responde lleva BLOQUE_UNIVERSAL y (salvo la recepcionista) BLOQUE_AGENTE, con alcance, acciones permitidas y prohibiciones. Hay controles en código: enmascarado de RUT antes de guardarlo, detección de instrucciones inyectadas en web snapshots y documentos, derivación de síntomas agudos con mensaje fijo y validaciones de argumentos por herramienta. El notebook muestra una línea de corrección real («recomendación retirada») y la respuesta declara límites. Las pruebas S1-S6 están especificadas, pero su corrida no está versionada.

5. Ampliaciones declaradas: análisis

El proyecto declara en spec §9 (DA-06) cuatro bonos por +3,0 y descarta el MCP porque no aporta al propósito (y la herramienta de acción no aplica: el intent prohíbe agendar). La selección es coherente con el problema y no duplica mecanismos.

RAG con el Redis del curso (+1,0) — sólido

`consultar_documentos` recupera por similitud usando embeddings de OpenAI (text-embedding-3-small, 768 dims) y un índice RediSearch del curso sobre hashes con prefijo `cotizador:frag`, campos `tag/texto` y vector, métrica coseno, algoritmo exacto FLAT, top-k=4 y filtros por centro y código. La ingesta es idempotente y está documentada (spec §5.9). El agente decide **cuándo** recuperar: el caso G11 lo activa y G01-G10 no lo usan. Se retiran precios del canal documental [P-04]. *Mejora sugerida*: agregar al golden set un caso explícito de «sin necesidad» (afirmar `no_debe_usar: consultar_documentos`) y citar la fuente en la respuesta.

Workflow adicional — router (+1,0) — cumple

El router clasifica en cuatro rutas (`cotizar`, `info_publicada`, `respuesta_directa`, `fuera_de_alcance`) y `main.ruta_condicional` decide por código el destino, con reinicio del estado del turno. Los casos G11, G14, G16 y las pruebas S1-S3 verifican la ruta y el uso (o no) de herramientas. Hay traza de la ruta seleccionada en cada corrida.

Evaluación con golden set (+0,5) — parcial

El golden set G01-G16 está versionado y es completo: cubre N por defecto, N indicado, menos centros que N, previsión desconocida, no coincide, no contesta, llamada cortada, intenta agendar, sin agenda, examen suspendido, info publicada, mismo nombre con distinto propósito, coincidencia dudosa, necesidad ambigua, examen inexistente y jailbreak. Cada caso declara solo lo comparable y `run_golden.py` genera un informe con hojas Detalle, Resumen y Fallos. **Falta lo que la pauta califica**: la corrida ejecutada y versionada. El repositorio no contiene ningún `resultados_*.xlsx`, la spec reclama 100 % en dos corridas (P-12) y el plan marca el paso como «hecho» sin artefacto. Si el revisor no ve ejecuciones, el bono puede no sumar.

Guardrail avanzado / juez — verificador (+0,5) — cumple

El verificador es un control independiente por código, por encima de la seguridad basal: comprueba que cada dato del reporte exista en una observación, y lo no respaldado pasa a «no confirmado»; también retira recomendaciones del texto. Hay caso benigno (aprobado sin cambios) y adversarial (precio inventado o «le recomiendo»). La corrida del notebook es evidencia real: aplicó «texto: recomendación retirada». *Mejora sugerida*: versionar la salida de la prueba benigna/adversarial del verificador.

6. Fortalezas

- **Arquitectura limpia y modular**: `main.py` sin lógica de negocio, nodos/herramientas/agentes/servicios/prompts separados, config como único lector del entorno.
- **Datos y mundo simulados, pero agentes reales**: la exactitud se mide contra la verdad del simulador, lo que hace el criterio de éxito verificable.
- **Seguridad integrada, no pegada**: bloques compartidos, enmascarado de RUT, anti-inyección y derivación de síntomas conviven con las validaciones de herramientas.
- **Trazabilidad de punta a punta**: observaciones en JSON, motivo de parada calculado en código y captura del paso a paso en el arnés de validación.
- **Documentación de decisiones**: spec con conflictos resueltos, decisiones abiertas, plan por pasos y prompts de desarrollo en OpenCode adjuntos por separado.
- **Pruebas de dos naturalezas**: golden set funcional (16 casos) y pruebas de seguridad (6), más pruebas de componente en la spec.

7. Brechas y riesgos

#	Brecha / riesgo	Impacto	Severidad
1	El notebook entregado ejecuta un solo turno; no demuestra el segundo turno del historial, que la pauta pide explícitamente.	Puede poner en riesgo el criterio de historial (0,5) y, si el revisor no ve el flujo, activar el tope de 3,0.	Alta
2	No hay corridas versionadas del golden set ni de seguridad (faltan resultados_*.xlsx). El checklist del notebook dice PENDIENTE.	Debilita el bono de golden set (0,5), parte del guardrail y la «verificación final» exigida antes de entregar.	Alta
3	Los bonos se declaran en spec (DA-06) pero no en el README ni en el notebook, con su ubicación de ejecución y prueba.	La pauta pide declarar los bonos y dónde se ejecutan; su ausencia en la entrega puede hacer que no se acrediten.	Media
4	La portada del notebook no enuncia el caso, el usuario, el alcance ni el criterio de éxito observable.	El criterio «caso y criterio de éxito» (0,5) depende de que el revisor lo encuentre en spec/README.	Media
5	El RAG no tiene un caso de «sin necesidad» que afirme que NO se recupera cuando no corresponde.	La pauta exige mostrar entradas con y sin necesidad de uso del corpus.	Media-baja
6	requirements.txt usa rangos (>=) en vez de versiones fijadas; algunos textos de la spec quedaron desalineados (menciona Gemini y un método de entrega distinto al implementado).	Reproducibilidad del entorno y confusión menor en la ficha técnica.	Baja
7	Texto heredado en responder_prompt: «la cotización aún no está disponible en esta versión».	Ruido de la etapa de placeholder; puede inducir respuestas confusas.	Baja

8. Plan de acción priorizado para asegurar el 7,0

Prioridad	Acción	Dónde
1	Ejecutar el golden set G01-G16 y las pruebas S1-S6, y versionar los informes (resultados_*.xlsx y seguridad_*.xlsx) con la corrida final en verde.	validacion/
2	Cambiar el notebook a la prueba de dos turnos de spec §7.3 («medición de glaucoma en Talca, soy Fonasa» y «¿Y un fondo de ojo?») y reejecutar mostrando la reutilización de ciudad/previsión. Que el checklist quede en PASÓ.	notebooks/prueba_cotizador.ipynb
3	Añadir una sección «Bonos declarados» al README con, para cada bono: mecanismo, dónde se ejecuta y prueba que lo respalda.	README.md y notebook
4	Poner en la portada del notebook el caso, usuario, entrada, alcance, salida esperada y el criterio de éxito observable.	notebooks/prueba_cotizador.ipynb
5	Agregar al golden set un caso con no_debe_usar: consultar_documentos y exigir fuente citada en G11.	validacion/golden_set.json
6	Fijar versiones en requirements.txt y corregir textos desalineados de la spec (Gemini, método de entrega) y el texto heredado del responder.	requirements.txt, spec.md, prompts/responder_prompt.py

9. Veredicto

El proyecto **satisface la base obligatoria** y es, con diferencia, un trabajo de nivel alto: el flujo central está completo, la seguridad es transversal y las cuatro ampliaciones están implementadas con criterio. No se observan fallas que obliguen al tope de 3,0: LLM, ReAct, herramienta, devolución de observación, parada e historial existen y están razonablemente probados.

La diferencia entre **~93 % y el 100 %** está casi por completo en la **evidencia de ejecución**, no en el código: falta versionar las corridas del golden set y de seguridad y falta que el notebook demuestre el segundo turno. Son ajustes de bajo esfuerzo y alto retorno; si se realizan los seis puntos del capítulo 8, la entrega queda en condiciones de pelear la nota máxima.

Escenario de calificación	Estimación
Solo con la evidencia versionada hoy (sin ejecutar pruebas)	~86 % (6,0 / 7,0)
El revisor ejecuta el golden set y la seguridad, y pasan (como afirma el plan)	~93 % (6,5 / 7,0)
Tras aplicar el plan de acción del capítulo 8 y reejecutar	~100 % (7,0 / 7,0)

Anexo. Correspondencia directa con la pauta: los cinco criterios de la base están cubiertos (capítulo 4) y los cuatro bonos declarados se corresponden con RAG, workflow, golden set y guardrail (capítulo 5). El entorno, los prompts y las pruebas se entregan como evidencia, sin secretos, con .env.example y datos ficticios versionados. Informe generado a partir de la pauta docs/tarea_final.pdf y del código del proyecto.